Reasoning About Vectors Using an SMT Theory of Sequences

Ying Sheng[1], Andres Noetzli[1], Andrew Reynolds[2], Yoni Zohar[3], David Dill[4], Wolfgang Grieskamp[4], Junkil Park[4], Shaz Qadeer[4], Clark Barrett[1], Cesare Tinelli[2]

[1]Stanford University

[2]The University of Iowa

[3]Bar-Ilan University

[4]Meta Novi

Structure of This Talk

- Pre-talk: General introduction to the field
- Talk: Describe a recent research project
- (Skip technical background – will fill in along the way)

Pre-talk

Why Formal Verification?

What Does This Code Do?

```
int b(int* a, int size, int key)
{
  int low = 0;
  int high = size - 1;
  while (low <= high)
  {
    int mid = (low + high) / 2;
    int midVal = a[mid];
    if (midVal < key)
    {
      low=mid+1;
    } else if (midVal > key)
    {
      high=mid-1;
    }
    else {
      return mid;
    }
    return -(low + 1);
  }
  return -1;
}
```

Why Formal Verification?

Where is The Bug in This Code?

```
int b(int* a, int size, int key)
{
  int low = 0;
  int high = size - 1;
  while (low <= high)
  {
    int mid = (low + high) / 2;
    int midVal = a[mid];
    if (midVal < key)
    {
      low=mid+1;
    } else if (midVal > key)
    {
      high=mid-1;
    }
    else {
      return mid;
    }
    return -(low + 1);
  }
  return -1;
}
```

Why Formal Verification?

Where is The Bug in This Code?

```
int b(int* a, int size, int key)
{
  int low = 0;
  int high = size - 1;
  while (low <= high)
  {
    int mid = (low + high) / 2;
    int midVal = a[mid];
    if (midVal < key)
    {
      low=mid+1;
    } else if (midVal > key)
    {
      high=mid-1;
    }
    else {
      return mid;
    }
    return -(low + 1);
  }
  return -1;
}
```

- Overflow
- int mid = (low + high) / 2;
- int mid = low + (high - low) / 2;

Bugs in Software

- Writing code is a complicated intellectual activity
- We are humans
- Our code has mistakes

Bugs in Software

- Writing code is a complicated intellectual activity
- We are humans
- Our code has mistakes

- Disclaimer: of course you do not have bugs. You are special.

Formal Verification: The Turning Point

Intel FDIV Bug



- 1994: Pentium Processors
- Bug in division operation
- Was found only after distribution
- Cost: $475M

Result: Massive Progress in Formal Verification

Formal Verification: More Recent Motivation

Blockchain Horror Stories



- Blockchain is a new technology for distributed finance
- In the beginning (Bitcoin): simple operations
- Nowadays (Ethereum): arbitrary complex smart contracts
- 2016: The DAO Bug – $40M worth of crypto was stolen
- Many similar bugs caused other losses

Formal Verification: More Recent Motivation

Boeing 737



- 2018,2019: Plane Crashes
- Many reasons, including software bugs

> Boeing says it has a software fix ready for its 737 Max airplanes that will be unveiled to airline officials, pilots and aviation authorities from around the world Wednesday, as the aircraft manufacturer works to rebuild trust among its customers and the flying public following two fatal crashes of the planes in recent months.

Formal Verification

Importance



- Software is everywhere
- Humans write it and make mistakes (Not you!)
- If we know of a bug: stop everything until fixed
- Dangerous: Unaware of bugs

How can we find bugs?

The Traditional Approach: Testing



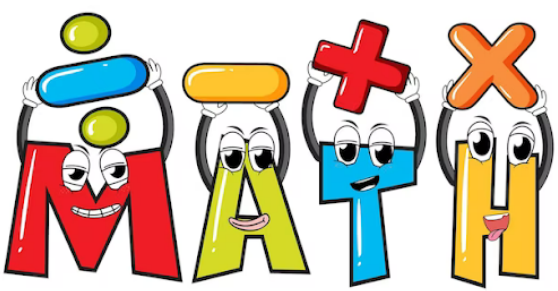
- Run your code on a range of inputs
- QA teams or programmers
- Partial Automation
- Bug . . .

We will never cover all cases

An Alternative Approach: Math

- In math: we want to prove that a theorem always holds
- In SW: we want to prove that a program always satisfies something
- Mathematicians do not settle for tests
  - Is Pythagoras Theorem true becuase we tried some examples?
- Why should we?

Idea: formulate a math theorem that specifies the correctness of a program, and prove it.





Proving

- Analogy to math only goes so far
- programs are huge compared to theorems
- programs change rapidly, theorems are static

Proofs should be done Automatically



Formal Verification Tools

- CBMC, JBMC (Oxford)
- Pono (Stanford)
- Boogie (Microsoft Research)
- Zelkova (Amazon Web Services)
- Move-prover (Stanford, Facebook)
- KeY (Universities of Chalmers, KIT, Darmstadt)
- Jasper Gold (Cadence)
- SeaHorn (University of Waterloo)
- Many many more...

Demo

Demo

```
void s(int* x, int* y) {
  int tmp = *x;
  *x = y;
  *y = tmp;
}
```

Example

- What does the above code do?

Demo

```
void s(int* x, int* y) {
  int tmp = *x;
  *x = y;
  *y = tmp;
}
```

Example

- What does the above code do?
- swaps

Demo

```
void s(int* x, int* y) {
  int tmp = *x;
  *x = y;
  *y = tmp;
}
```

Example

- What does the above code do?
- swaps
- But it has a bug: where?

Demo

```
void s(int* x, int* y) {
  int tmp = *x;
  *x = y;
  *y = tmp;
}
```

Example

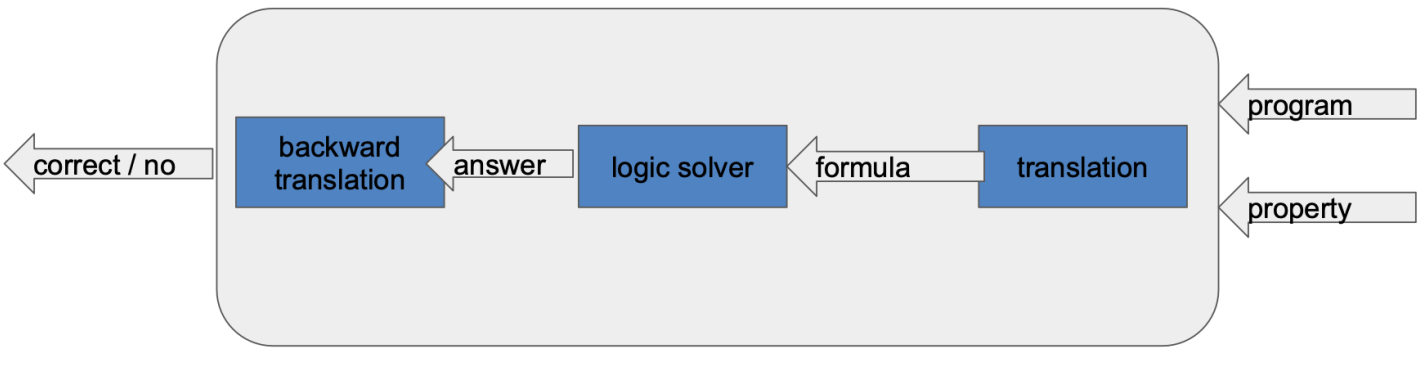
- What does the above code do?
- swaps
- But it has a bug: where?
- *y* is missing a $*$

Demo

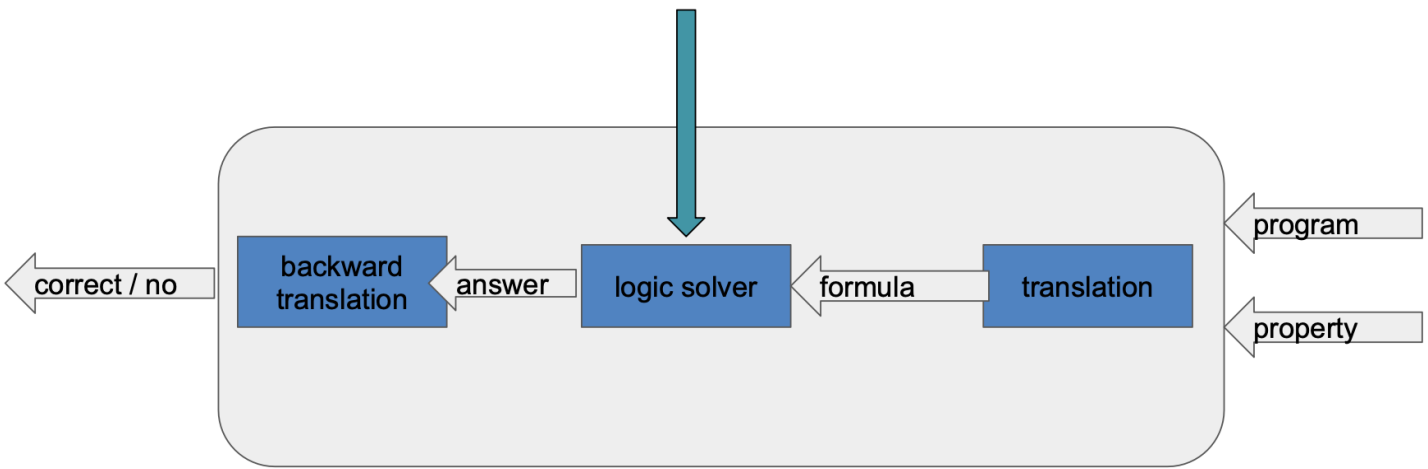
- Tool: SeaHorn
- sat means there is a bug
- unsat means there is no bug



How Does This Work?



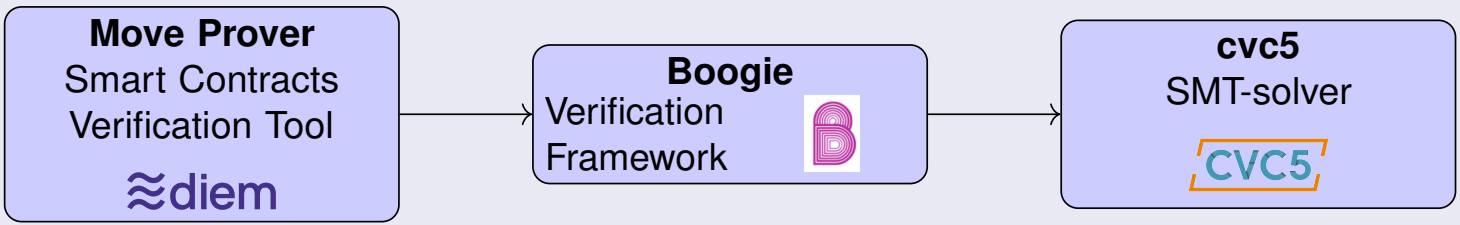
How Does This Work?



Talk

Smart Contract Verification & Move Prover

Motivation



Applications of Vectors

1. Encoding of the state (tree-shaped)
2. Modeling Move built-in `vector` data structure
3. Used in many programming languages

Desired Properties

1. Expressiveness: Supporting common operations
2. Generality: Generic vectors
3. Efficiency: Fast and efficient reasoning tool

Arrays vs. Sequences

Arrays

- ✗ Expressiveness: Only read and write
- ✓ Generality: The theory of arrays is generic
- ✓ Efficiency: There are efficient procedures

Arrays + UF + Quantifiers

- ✓ Expressiveness: Quantifieres can be used for axiomatization
- ✓ Generality: Can capture general elelemnt sorts
- ✗ Efficiency: Quantifiers reduce efficiency and stability

Sequences (this work)

- ✓ Expressiveness: Designed to support common operators
- ✓ Generality: Completely generic
- ✓ Efficiency: Leverages strings and arrays

The Theory of Sequences

Arithmetic Operators

Symbol	Arity	SMT-LIB
n	Int	n
$+$	$\text{Int} \times \text{Int} \to \text{Int}$	+
$-$	$\text{Int} \to \text{Int}$	-
$\leqslant$	$\text{Int} \times \text{Int} \to \text{Bool}$	<=

Sequence Operators

Symbol	Arity	SMT-LIB
ϵ	Seq	`seq.empty`
unit	$\text{Elem} \to \text{Seq}$	`seq.unit`
$\lvert_\rvert$	$\text{Seq} \to \text{Int}$	`seq.len`
nth	$\text{Seq} \times \text{Int} \to \text{Elem}$	`seq.nth`
update	$\text{Seq} \times \text{Int} \times \text{Elem} \to \text{Seq}$	`seq.update`
extract	$\text{Seq} \times \text{Int} \times \text{Int} \to \text{Seq}$	`seq.extract`
$_ +\!\!+ \cdots +\!\!+ _$	$\text{Seq} \times \cdots \times \text{Seq} \to \text{Seq}$	`seq.concat`

Comparison Example

```
// @pre: 0 <= i,j < s.size() and s[i] == s[j]
// @post: s_out == s
void swap(std::vector<int>& s, int i, int j) {
  int a = s[i];
  int b = s[j];
  s[i] = b;
  s[j] = a;
}
```

	Arrays
Problem Variables	a, b, i, j : Int $\quad s, s_{out} : V$
Auxiliary Variables	$\ell : V \to \text{Int} \quad \mathbf{c} : V \to \text{Arr}$ $\approx_{\mathbf{A}} : V \times V \to \text{Bool}$ $\text{nth}_{\mathbf{A}} : V \times \text{Int} \to \text{Int}$ $\text{update}_{\mathbf{A}} : V \times \text{Int} \times \text{Int} \to V$
Axioms	$\forall x, y. x \approx_{\mathbf{A}} y \leftrightarrow (\ell(x) \approx \ell(y) \wedge \forall\, 0 \leqslant i < \ell(x). \mathbf{c}(x)[i] \approx \mathbf{c}(y)[i])$ $\forall x, y, i, a. y \approx_{\mathbf{A}} \text{update}_{\mathbf{A}}(x, i, a) \leftrightarrow (\ell(x) \approx \ell(y) \wedge (0 \leqslant i < \ell(x) \to \mathbf{c}(y) \approx \mathbf{c}(x)[i \leftarrow a]))$
Program	$a \approx \text{nth}_{\mathbf{A}}(s, i) \wedge b \approx \text{nth}_{\mathbf{A}}(s, j)$ $s_{out} \approx_{\mathbf{A}} \text{update}_{\mathbf{A}}(\text{update}_{\mathbf{A}}(s, i, b), j, a)$
Spec.	$0 \leqslant i, j < \ell(s) \wedge \text{nth}_{\mathbf{A}}(s, i) \approx \text{nth}_{\mathbf{A}}(s, j)$ $\neg s_{out} \approx_{\mathbf{A}} s$

Comparison Example

```
// @pre: 0 <= i,j < s.size() and s[i] == s[j]
// @post: s_out == s
void swap(std::vector<int>& s, int i, int j) {
  int a = s[i];
  int b = s[j];
  s[i] = b;
  s[j] = a;
}
```

	Sequences
Problem Variables	a, b, i, j : Int $\quad s, s_{out}$: Seq
Auxiliary Variables	
Axioms	
Program	$a \approx \text{nth}(s, i) \wedge b \approx \text{nth}(s, j)$ $s_{out} \approx \text{update}(\text{update}(s, i, b), j, a)$
Spec.	$0 \leqslant i, j < \lvert s \rvert \wedge \text{nth}(s, i) \approx \text{nth}(s, j)$ $\neg s_{out} \approx s$

SMT-based Calculi for Sequences

Framework

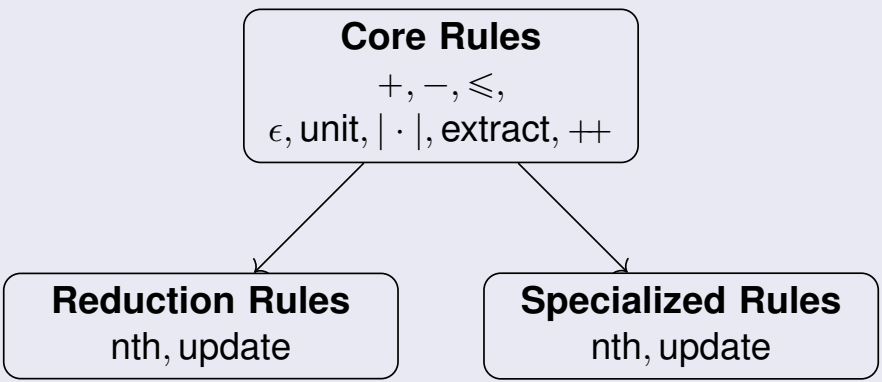
- Procedures are presented as calculi
- cvc5 implements a strategy of proof-search
- Not a decision procedure
  - Decidability is unknown

Calculi

- Basic calculus
  - Leverages techniques for the theory of strings
  - Lifting rules from characters to arbitrary element types
  - Eliminates nth and update
- Extended calculus
  - Same core rules
  - Array-like reasoning for nth and update

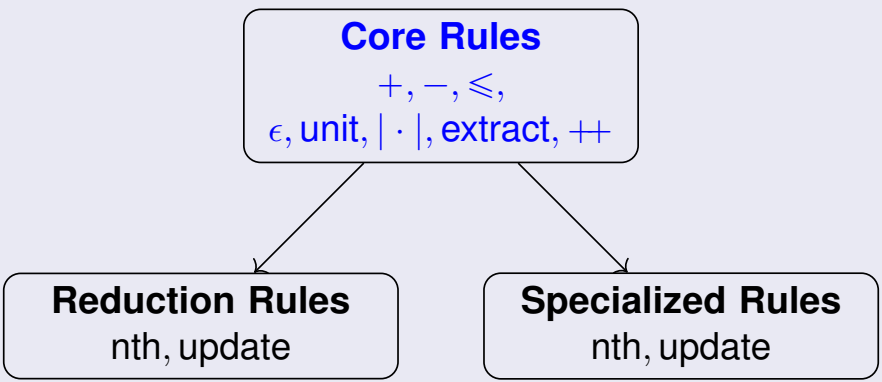
Calculi and Rules

Rules



Calculi and Rules

Rules



Atithmetic and Equality

- S: Seq-constraints ($\approx, \not\approx$)
- A: Arith-constraints (Arith terms)
- $\ell_x \approx |x|$

- $\mathcal{T}(\mathsf{S})$ – terms in S
- $\models_{\mathsf{LIA}}$: LIA-entailed
- $\models$: UF-entailed

Conflicts

$$\text{A-Conf}\ \frac{\mathsf{A} \models_{\mathsf{LIA}} \bot}{\text{unsat}} \qquad \text{S-Conf}\ \frac{\mathsf{S} \models \bot}{\text{unsat}}$$

Propagation

$$\text{A-Prop}\ \frac{\mathsf{A} \models_{\mathsf{LIA}} s \approx t \quad s,t \in \mathcal{T}(\mathsf{S})}{\mathsf{S} := \mathsf{S}, s \approx t}$$

$$\text{S-Prop}\ \frac{\mathsf{S} \models s \approx t \quad s,t \in \mathcal{T}(\mathsf{S}) \quad s,t \text{ are } \Sigma_{\mathsf{LIA}}\text{-terms}}{\mathsf{A} := \mathsf{A}, s \approx t}$$

$$\text{S-A}\ \frac{x,y \in \mathcal{T}(\mathsf{S}) \cap \mathcal{T}(\mathsf{A}) \quad x,y : \mathsf{Int}}{\mathsf{A} := \mathsf{A}, x \approx y \quad \| \quad \mathsf{A} := \mathsf{A}, x \not\approx y}$$

Reduced Form

- S: Seq-constraints ($\approx, \not\approx$)
- A: Arith-constraints (Arith terms)
- $\ell_x \approx |x|$

- $\mathcal{T}(\mathsf{S})$ – terms in S
- $\models_{\mathsf{LIA}}$: LIA-entailed
- $\models$: UF-entailed

Rule

$$\text{L-Intro}\ \frac{s \in \mathcal{T}(\mathsf{S}) \quad s : \mathsf{Seq}}{\mathsf{S} := \mathsf{S}, |s| \approx (|s|)\downarrow}$$

Rewrites

$|\epsilon| \to 0$ $\qquad$ $|\mathsf{unit}(t)| \to 1$

$|\mathsf{update}(s, i, t)| \to |s|$ $\qquad$ $|s_1 +\!\!+ \ldots +\!\!+ s_n| \to |s_1| + \ldots + |s_n|$

$\overline{u} +\!\!+ \epsilon +\!\!+ \overline{v} \to \overline{u} +\!\!+ \overline{v}$ $\qquad$ $\overline{u} +\!\!+ (s_1 +\!\!+ \ldots +\!\!+ s_n) +\!\!+ \overline{v} \to \overline{u} +\!\!+ s_1 +\!\!+ \ldots +\!\!+ s_n +\!\!+ \overline{v}$

Empty Sequence, Unit Sequence, Extentionality

- S: Seq-constraints ($\approx, \not\approx$)
- A: Arith-constraints (Arith terms)
- $\ell_x \approx |x|$

- $\mathcal{T}(\mathsf{S})$ – terms in S
- $\models_{\mathsf{LIA}}$: LIA-entailed
- $\models$: UF-entailed

Emptyness

$$\text{L-Valid}\ \frac{x \in \mathcal{T}(\mathsf{S}) \quad x : \mathsf{Seq}}{\mathsf{S} := \mathsf{S}, x \approx \epsilon \quad \| \quad \mathsf{A} := \mathsf{A}, \ell_x > 0}$$

Unit

$$\text{U-Eq}\ \frac{\mathsf{S} \models \mathsf{unit}(x) \approx \mathsf{unit}(y)}{\mathsf{S} := \mathsf{S}, x \approx y}$$

Extentionality

$$\text{Deq-Ext}\ \frac{x \not\approx y \in \mathsf{S} \quad x, y : \mathsf{Seq}}{\begin{array}{c}\mathsf{A} := \mathsf{A}, \ell_x \not\approx \ell_y \quad \| \\ \mathsf{A} := \mathsf{A}, \ell_x \approx \ell_y, 0 \leqslant i < \ell_x \quad \mathsf{S} := \mathsf{S}, w_1 \approx \mathsf{nth}(x, i), w_2 \approx \mathsf{nth}(y, i), w_1 \not\approx w_2\end{array}}$$

Extraction

- S: Seq-constraints ($\approx, \not\approx$)
- A: Arith-constraints (Arith terms)
- $\ell_x \approx |x|$

- $\mathcal{T}(\mathsf{S})$ – terms in S
- $\models_{\mathsf{LIA}}$: LIA-entailed
- $\models$: UF-entailed

Extraction Elimination

R-Extract
$$\frac{x \approx \mathsf{extract}(y, i, j) \in \mathsf{S}}{\begin{array}{c}\mathsf{A} := \mathsf{A}, i < 0 \vee i \geqslant \ell_y \vee j \leqslant 0 \qquad \mathsf{S} := \mathsf{S}, x \approx \epsilon \quad \| \\ \mathsf{A} := \mathsf{A}, 0 \leqslant i < \ell_y, j > 0, \ell_k \approx i, \ell_x \approx \min(j, \ell_y - i), \ell_{k'} \approx \ell_y - \ell_x - i \\ \mathsf{S} := \mathsf{S}, y \approx k +\!\!+ x +\!\!+ k'\end{array}}$$

- extract is eliminated using $+\!\!+$ and fresh variables
- Main case: $x \approx \mathsf{extract}(y, i, j)$ iff $y \approx k +\!\!+ x +\!\!+ k'$
- Other corner cases are handled

Concatenation: Normal Form

Definition

$x_1 \mathbin{+\!\!+} \ldots \mathbin{+\!\!+} x_n$ is singular in S if $S \models x_i \approx \epsilon$ for all, except at most one, variables $x_i, i \in [1, n]$.

Definition

1. $S \models_{++} x \approx x$ for all sequence variables $x \in \mathcal{T}(S)$.
2. $S \models_{++} x \approx t$ for all sequence variables $x \in \mathcal{T}(S)$ and variable concatenation terms t, where $x \approx t \in S$.
3. If $S \models_{++} x \approx (\overline{\boldsymbol{w}} \mathbin{+\!\!+} y \mathbin{+\!\!+} \overline{\boldsymbol{z}})\downarrow$ and $S \models y \approx t$ and t is ϵ or a variable concatenation term in S that is not singular in S, then $S \models_{++} x \approx (\overline{\boldsymbol{w}} \mathbin{+\!\!+} t \mathbin{+\!\!+} \overline{\boldsymbol{z}})\downarrow$.

Intuition

- $S \models_{++} x \approx t$ if x can be expanded to t via equalities in S.
- Each component of t is singular.

Concatanation: Atomic Representatives

Definition

x is atomic in S if $S \not\models x \approx \epsilon$ and for all variable concatenation terms $s \in \mathcal{T}(S)$ such that $S \models x \approx s$, s is singular in S.

Definition

$x \equiv_S y$ iff $S \models x \approx y$. From each eq. class, we choose a representative.

Definition

$S \models^*_{++} x \approx \overline{\boldsymbol{y}}\ \overline{\boldsymbol{y}}$ consists of atomic representatives and there exists $\overline{\boldsymbol{z}}$ of such that $S \models_{++} x \approx \overline{\boldsymbol{z}}$ and $S \models y_i \approx z_i$.

Intuition

$S \models^*_{++} x \approx t$ holds when t is a concatenation of atomic representatives.

Normal Form

$\overline{\boldsymbol{t}}$ is the normal form of x.

Concatenation: Rules

- S: Seq-constraints ($\approx, \not\approx$)
- A: Arith-constraints (Arith terms)
- $\ell_x \approx |x|$

- $\mathcal{T}(\mathsf{S})$ – terms in S
- $\models_{\mathsf{LIA}}$: LIA-entailed
- $\models$: UF-entailed

Unifying

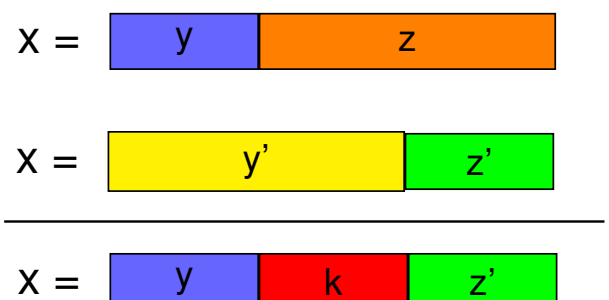
$$\text{C-Eq}\ \frac{\mathsf{S} \models^*_{++} x \approx \overline{z} \qquad \mathsf{S} \models^*_{++} y \approx \overline{z}}{\mathsf{S} := \mathsf{S}, x \approx y}$$

Splitting

$$\text{C-Split}\ \frac{\mathsf{S} \models^*_{++} x \approx (\overline{w} +\!\!+ y +\!\!+ \overline{z})\downarrow \qquad \mathsf{S} \models^*_{++} x \approx (\overline{w} +\!\!+ y' +\!\!+ \overline{z'})\downarrow}{\begin{array}{lll} \mathsf{A} := \mathsf{A}, \ell_y > \ell_{y'} & \mathsf{S} := \mathsf{S}, y \approx y' +\!\!+ k & \| \\ \mathsf{A} := \mathsf{A}, \ell_y < \ell_{y'} & \mathsf{S} := \mathsf{S}, y' \approx y +\!\!+ k & \| \\ \mathsf{A} := \mathsf{A}, \ell_y \approx \ell_{y'} & \mathsf{S} := \mathsf{S}, y \approx y' & \end{array}}$$

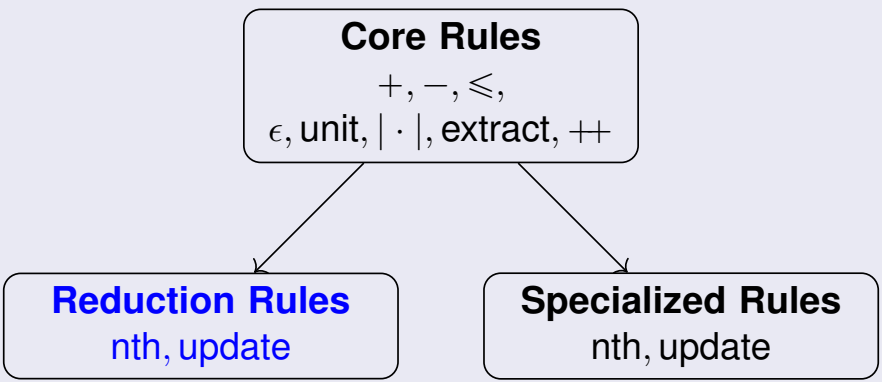
Example of Splitting

$$\text{C-Split}\ \frac{\mathsf{S} \models^{*}_{++} x \approx (\overline{\boldsymbol{w}} +\!\!+ y +\!\!+ \overline{\boldsymbol{z}})\!\downarrow \qquad \mathsf{S} \models^{*}_{++} x \approx (\overline{\boldsymbol{w}} +\!\!+ y' +\!\!+ \overline{\boldsymbol{z}'})\!\downarrow}{\begin{array}{c}\mathsf{A} := \mathsf{A}, \ell_y > \ell_{y'} \qquad \mathsf{S} := \mathsf{S}, y \approx y' +\!\!+ k \quad \| \\ \mathsf{A} := \mathsf{A}, \ell_y < \ell_{y'} := \mathsf{S}, y' \approx y +\!\!+ k \quad \| \\ \mathsf{A} := \mathsf{A}, \ell_y \approx \ell_{y'} \qquad \mathsf{S} := \mathsf{S}, y \approx y'\end{array}}$$



Calculi and Rules

Rules



nth, update

- S: Seq-constraints ($\approx, \not\approx$)
- A: Arith-constraints (Arith terms)
- $\ell_x \approx |x|$

- $\mathcal{T}(\mathsf{S})$ – terms in S
- $\models_{\mathsf{LIA}}$: LIA-entailed
- $\models$: UF-entailed

Reduction Rules

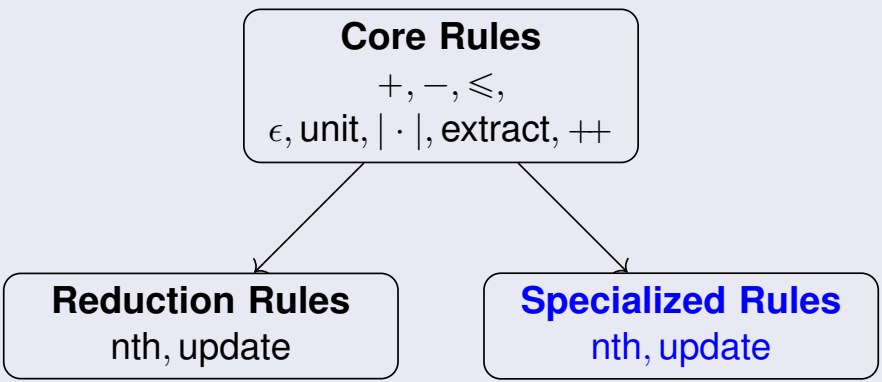
$$\text{R-Nth}\ \frac{x \approx \mathsf{nth}(y, i) \in \mathsf{S}}{\mathsf{A} := \mathsf{A}, i < 0 \vee i \geqslant \ell_y \quad \| \quad \mathsf{A} := \mathsf{A}, 0 \leqslant i < \ell_y, \ell_k \approx i \quad \mathsf{S} := \mathsf{S}, y \approx k +\!\!+ \mathsf{unit}(x) +\!\!+ k'}$$

$$\text{R-Update}\ \frac{x \approx \mathsf{update}(y, i, z) \in \mathsf{S}}{\mathsf{A} := \mathsf{A}, i < 0 \vee i \geqslant \ell_y \quad \mathsf{S} := \mathsf{S}, x \approx y \quad \| \quad \mathsf{A} := \mathsf{A}, 0 \leqslant i < \ell_y, \ell_k \approx i, \ell_{k'} \approx 1 \quad \mathsf{S} := \mathsf{S}, y \approx k +\!\!+ k' +\!\!+ k'', x \approx k +\!\!+ \mathsf{unit}(z) +\!\!+ k''}$$

- nth and update are eliminated using fresh variables and $+\!\!+$
- Main case: $e \approx \mathsf{nth}(s, i)$ iff $s \approx s +\!\!+ \mathsf{unite} +\!\!+ s'$
- OOB cases

Calculi and Rules

Rules



Array Reasoning

- S: Seq-constraints ($\approx, \not\approx$)
- A: Arith-constraints (Arith terms)
- $\ell_x \approx |x|$

- $\mathcal{T}(\mathsf{S})$ – terms in S
- $\models_{\mathsf{LIA}}$: LIA-entailed
- $\models$: UF-entailed

Read Over Write

Nth-Update

$$\frac{\mathsf{nth}(x,j) \in \mathcal{T}(\mathsf{S}) \qquad y \approx \mathsf{update}(z,i,v) \in \mathsf{S} \qquad \mathsf{S} \models x \approx y \text{ or } \mathsf{S} \models x \approx z}{\begin{array}{l} \mathsf{A} := \mathsf{A}, j < 0 \vee j \geqslant \ell_x \quad \| \\ \mathsf{A} := \mathsf{A}, i \approx j, 0 \leqslant j < \ell_x \qquad \mathsf{S} := \mathsf{S}, \mathsf{nth}(y,j) \approx v \quad \| \\ \mathsf{A} := \mathsf{A}, i \not\approx j, 0 \leqslant j < \ell_x \qquad \mathsf{S} := \mathsf{S}, \mathsf{nth}(y,j) \approx \mathsf{nth}(z,j) \end{array}}$$

- Variant of read-over-write axioms
- Takes into account OOB
- Within bounds: updated element changes, the rest remain the same
- Out of bounds: no effect

Unit + nth + update

- S: Seq-constraints ($\approx, \not\approx$)
- A: Arith-constraints (Arith terms)
- $\ell_x \approx |x|$

- $\mathcal{T}(\mathsf{S})$ – terms in S
- $\models_{\mathsf{LIA}}$: LIA-entailed
- $\models$: UF-entailed

unit + nth

$$\textbf{Nth-Unit}\ \frac{x \approx \mathsf{nth}(y, i) \in \mathsf{S} \quad \mathsf{S} \models y \approx \mathsf{unit}(u)}{\mathsf{A} := \mathsf{A}, i < 0 \vee i > 0 \quad \| \quad \mathsf{A} := \mathsf{A}, i \approx 0 \quad \mathsf{S} := \mathsf{S}, x \approx u}$$

unit + update

$$\textbf{Update-Unit}\ \frac{x \approx \mathsf{update}(y, i, v) \in \mathsf{S} \quad \mathsf{S} \models y \approx \mathsf{unit}(u)}{\begin{array}{c}\mathsf{A} := \mathsf{A}, i < 0 \vee i > 0 \quad \mathsf{S} := \mathsf{S}, x \approx \mathsf{unit}(u) \quad \| \\ \mathsf{A} := \mathsf{A}, i \approx 0 \quad \mathsf{S} := \mathsf{S}, x \approx \mathsf{unit}(v)\end{array}}$$

$+\!\!+$ + nth + update

- S: Seq-constraints ($\approx, \not\approx$)
- A: Arith-constraints (Arith terms)
- $\ell_x \approx |x|$

- $\mathcal{T}(\mathsf{S})$ – terms in S
- $\models_{\mathsf{LIA}}$: LIA-entailed
- $\models$: UF-entailed

$+\!\!+$ + nth

Nth-Concat

$$\frac{x \approx \mathrm{nth}(y, i) \in \mathsf{S} \quad \mathsf{S} \models^{*}_{+\!+} y \approx w_1 +\!\!+ \ldots +\!\!+ w_n}{\mathsf{A} := \mathsf{A}, i < 0 \vee i \geqslant \ell_y \quad \| \quad \mathsf{A} := \mathsf{A}, 0 \leqslant i < \ell_{w_1} \quad \mathsf{S} := \mathsf{S}, x \approx \mathrm{nth}(w_1, i) \quad \| \quad \ldots \quad \| \quad \mathsf{A} := \mathsf{A}, \sum_{j=1}^{n-1} \ell_{w_j} \leqslant i < \sum_{j=1}^{n} \ell_{w_j} \quad \mathsf{S} := \mathsf{S}, x \approx \mathrm{nth}(w_n, i - \sum_{j=1}^{n-1} \ell_{w_j})}$$

$+\!\!+$ + update

Update-Concat

$$\frac{x \approx \mathrm{update}(y, i, v) \in \mathsf{S} \quad \mathsf{S} \models^{*}_{+\!+} y \approx w_1 +\!\!+ \ldots +\!\!+ w_n}{\mathsf{S} := \mathsf{S}, x \approx z_1 +\!\!+ \ldots +\!\!+ z_n, z_1 \approx \mathrm{update}(w_1, i, v), \ldots, z_n \approx \mathrm{update}(w_n, i - \sum_{j=1}^{n-1} \ell_{w_j}, v)}$$

Update-Concat-Inv

$$\frac{x \approx \mathrm{update}(y, i, v) \in \mathsf{S} \quad \mathsf{S} \models^{*}_{+\!+} x \approx w_1 +\!\!+ \ldots +\!\!+ w_n}{\mathsf{S} := \mathsf{S}, y \approx z_1 +\!\!+ \ldots +\!\!+ z_n, \quad w_1 \approx \mathrm{update}(z_1, i, v), \ldots, w_n \approx \mathrm{update}(z_n, i - \sum_{j=1}^{n-1} \ell_{w_j}, v)}$$

Additional Rules

- S: Seq-constraints ($\approx, \not\approx$)
- A: Arith-constraints (Arith terms)
- $\ell_x \approx |x|$

- $\mathcal{T}(\mathsf{S})$ – terms in S
- $\models_{\mathsf{LIA}}$: LIA-entailed
- $\models$: UF-entailed

Additional Rules

$$\textbf{Nth-Intro}\ \frac{s' \approx \mathsf{update}(s, i, t) \in \mathsf{S}}{\mathsf{S} := \mathsf{S}, e \approx \mathsf{nth}(s, i), e' \approx \mathsf{nth}(s', i)}$$

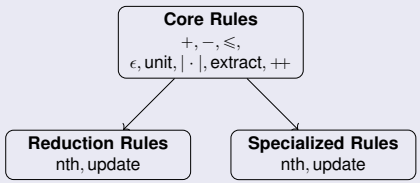
$$\textbf{Update-Bound}\ \frac{x \approx \mathsf{update}(y, i, v) \in \mathsf{S}}{\mathsf{A} := \mathsf{A}, 0 \leqslant i < \ell_y \qquad \mathsf{S} := \mathsf{S}, \mathsf{nth}(y, i) \not\approx v \quad \| \quad \mathsf{S} := \mathsf{S}, x \approx y}$$

$$\textbf{Nth-Split}\ \frac{\mathsf{nth}(x, i), \mathsf{nth}(x', i') \in \mathcal{T}(\mathsf{S}) \quad i \approx i' \in \mathsf{A}}{\mathsf{S} := \mathsf{S}, x \approx x' \quad \| \quad \mathsf{S} := \mathsf{S}, x \not\approx x'}$$

Correctness

Algorithm

1. Partition to Seq and Arith constraints
2. Apply all rules until no rule applies
3. If got ‘unsat’, return ‘unsat’. Else return ‘sat’



Termination

The above algorithm does not always terminate (e.g., $x +\!+ y \approx y +\!+ x$).

Theorem

When the above algorithm terminates, its result is correct.

proof

- unsat: simple local soundness check.
- sat: weak equivalence graph and normal forms.

Model Construction – Basics

Sorts

- Elem is some infinite set
- Seq and Int are pre-determined

Theory Symbols

- All but OOB nth are pre-determined
- OOB nth is handled at a later stage

Integer Variables

- There must be some integer model
- Use that model

$$\text{A-Conf}\ \frac{A \models_{\text{LIA}} \bot}{\text{unsat}}$$

Element Variables

- Chosen arbitrarily according to the CC of sequence constraints
- Possible since the domain is infinite

Model Construction – Sequences

Atomic Sequence Variables

- length: According to ℓ_x variables (assigned by the arithmetic model)
- unit variables x: means that $x \approx \text{unit}(e)$ is entailed and e was already assigned
- non-unit variables: weakly equivalent arrays [Christ & Hoenicke 2015]

Non-atomic Sequence Variables

- Transformation to normal form
- Concatenation of atomic variables (already assigned)

Theory vs. Practice

From Theory to Practice

- Got stuck with a case in the proof
- (updating a non-atomic variable)
- Create a benchmark that represents the case
- Found a wrong model in the implementation

And Back

- Got stuck in another case in the proof
- Checked the implementation
- Found out that it splits more equalities than we have in the calculus
- fixed the calculus and the proof

Evaluation

Implementation

- Both calculi are Implemented in cvc5
- Extension of an existing string solver

Benchmarks

- Smart Contracts Verification (558)
  - Real-world
  - Generated by the Move Prover
- Array benchmarks (551)
  - Translated from SMT-LIB QF_AX
  - Only update and nth

Tools

- *cvc*5: Basic calculus + Extended calculus
- Z3

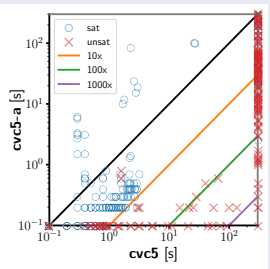
Results

Overall Resutls

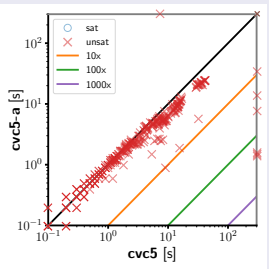
Set		w/ update cvc5	w/ update cvc5-a	w/ update Z3
ARRAYS (551)	Slvd	242	**390**	170
	Time	162	303	4329
DIEM (558)	Slvd	542	**547**	443
	Time	518	440	639

Commonly Solved

Arrays



SC Verification



- cvc5: core calculus + reduction rules
- cvc5-a: core calculus + specialized rules
- Z3: updates were eagerly eliminated
- 300s timeout

- cvc5-a solves the highest number of benchmarks
- Commonly solved: cvc5 5 is usually faster

Conclusion

We Have Seen

- New theory for reasoning about vectors
  - ✓ Expressive
  - ✓ General
  - ✓ Efficient
- Two Calculi
- Evaluation: efficient on real-world + crafted benchmarks

Future Work

- Standardize
- More array-like reasoning
- Decidability of fragments
- Polite theory combination

Conclusion

We Have Seen

- New theory for reasoning about vectors
  - ✓ Expressive
  - ✓ General
  - ✓ Efficient
- Two Calculi
- Evaluation: efficient on real-world + crafted benchmarks

Future Work

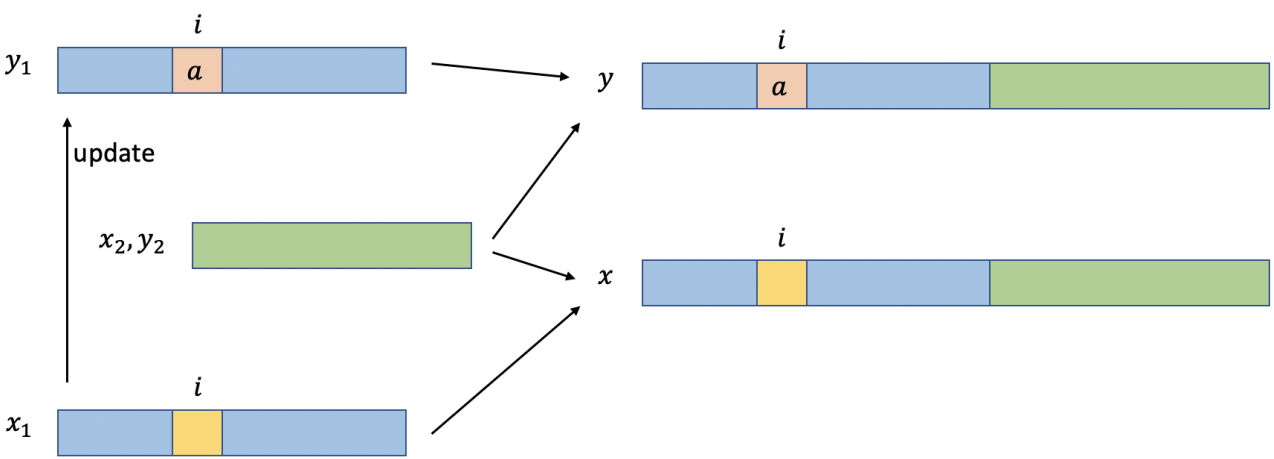
- Standardize
- More array-like reasoning
- Decidability of fragments
- Polite theory combination

Thank You!

Back Up Slides

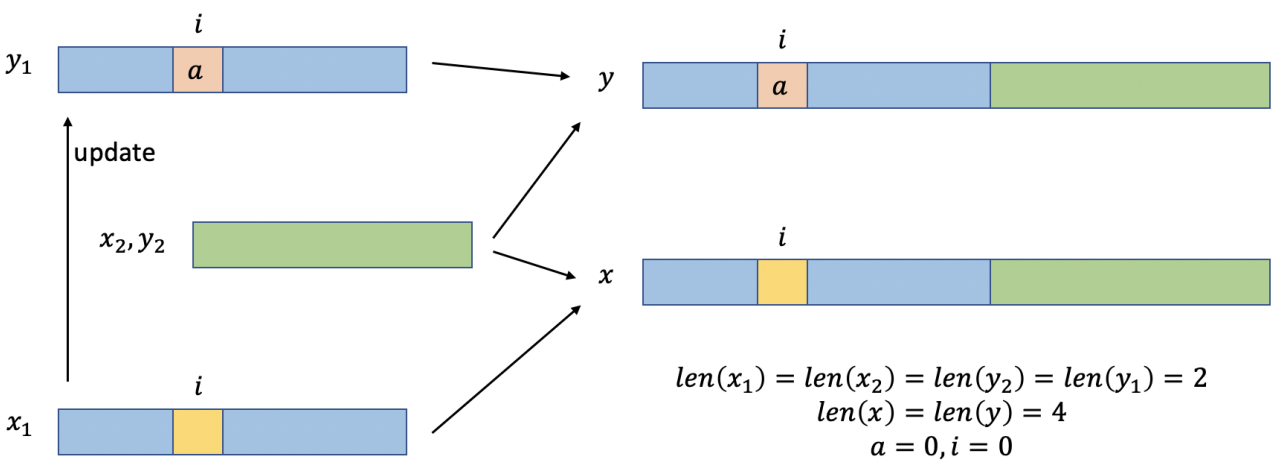
Model Construction - a very simple example

- Initial Configuration:
  $y = \text{update}(x, i, a), y = y_1 +\!\!+ y_2 \in \mathsf{S}_0$
  $0 \leqslant i < |y_1|, |y_1| > 0, |y_2| > 0 \in \mathsf{A}_0$.
- Saturated Configuration: $y \approx y_1 +\!\!+ y_2$, $x \approx x_1 +\!\!+ x_2$, $y_2 \approx x_2$, $y_1 \approx \text{update}(x_1, i, a)$, $|y_1| = |x_1|$, $|y_2| = |x_2|$, $\text{nth}(y, i) \approx a$, $\text{nth}(y_1, i) \approx a$, plus the original constraints.



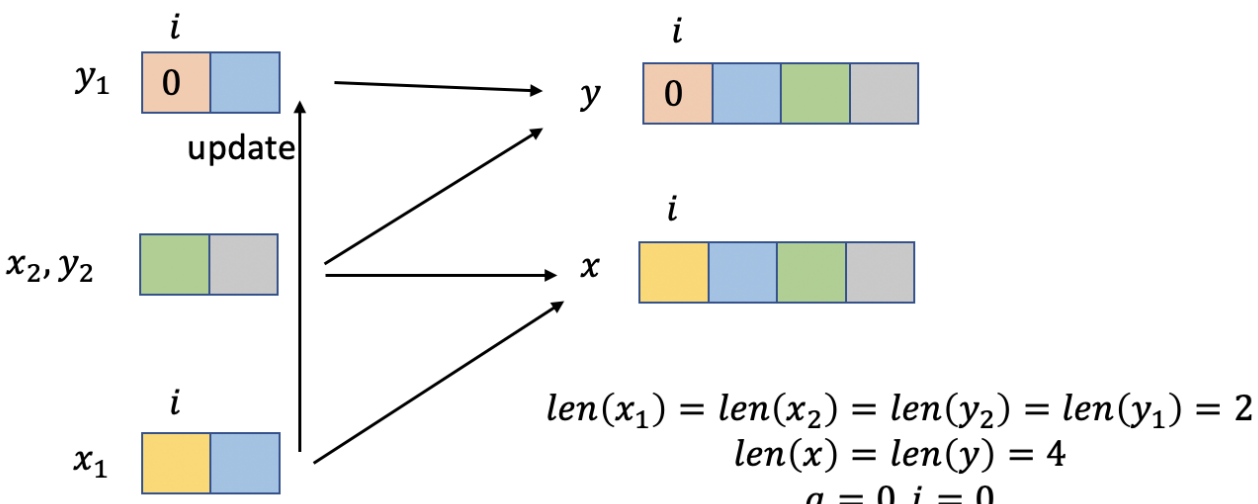
Model Construction - a very simple example

- Initial Configuration:
  $y = \text{update}(x, i, a), y = y_1 +\!\!+ y_2 \in S_0$
  $0 \leqslant i < |y_1|, |y_1| > 0, |y_2| > 0 \in A_0$.
- Saturated Configuration: $y \approx y_1 +\!\!+ y_2$, $x \approx x_1 +\!\!+ x_2$, $y_2 \approx x_2$, $y_1 \approx \text{update}(x_1, i, a)$, $|y_1| = |x_1|$, $|y_2| = |x_2|$, $\text{nth}(y, i) \approx a$, $\text{nth}(y_1, i) \approx a$, plus the original constraints.



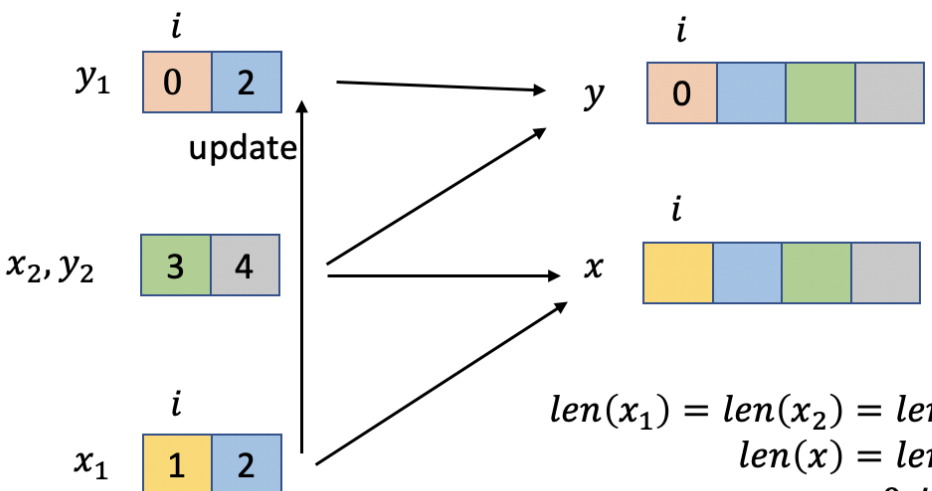
Model Construction - a very simple example

- Initial Configuration:
  $y = \text{update}(x, i, a), y = y_1 +\!\!+ y_2 \in S_0$
  $0 \leqslant i < |y_1|, |y_1| > 0, |y_2| > 0 \in A_0$.
- Saturated Configuration: $y \approx y_1 +\!\!+ y_2$, $x \approx x_1 +\!\!+ x_2$, $y_2 \approx x_2$, $y_1 \approx \text{update}(x_1, i, a)$, $|y_1| = |x_1|$, $|y_2| = |x_2|$, $\text{nth}(y, i) \approx a$, $\text{nth}(y_1, i) \approx a$, plus the original constraints.



Model Construction - a very simple example

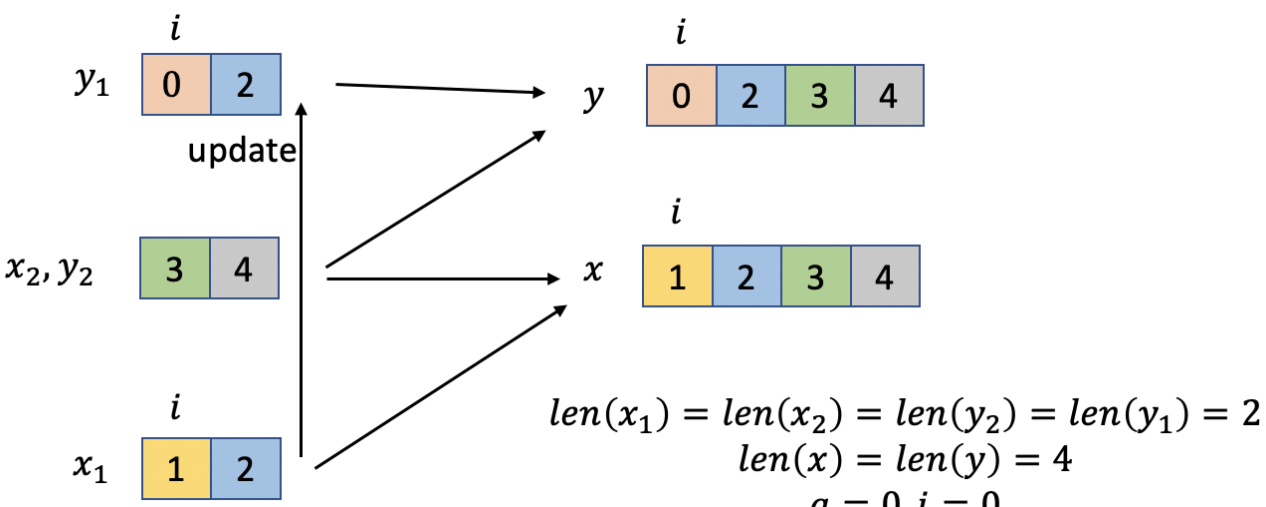
- Initial Configuration:
  $y = \text{update}(x, i, a), y = y_1 +\!\!+ y_2 \in S_0$
  $0 \leqslant i < |y_1|, |y_1| > 0, |y_2| > 0 \in A_0$.
- Saturated Configuration: $y \approx y_1 +\!\!+ y_2$, $x \approx x_1 +\!\!+ x_2$, $y_2 \approx x_2$, $y_1 \approx \text{update}(x_1, i, a)$, $|y_1| = |x_1|$, $|y_2| = |x_2|$, $\text{nth}(y, i) \approx a$, $\text{nth}(y_1, i) \approx a$, plus the original constraints.



$$len(x_1) = len(x_2) = len(y_2) = len(y_1) = 2$$
$$len(x) = len(y) = 4$$
$$a = 0, i = 0$$

Model Construction - a very simple example

- Initial Configuration:
  $y = \text{update}(x, i, a), y = y_1 +\!\!+ y_2 \in S_0$
  $0 \leqslant i < |y_1|, |y_1| > 0, |y_2| > 0 \in A_0$.
- Saturated Configuration: $y \approx y_1 +\!\!+ y_2$, $x \approx x_1 +\!\!+ x_2$, $y_2 \approx x_2$, $y_1 \approx \text{update}(x_1, i, a)$, $|y_1| = |x_1|$, $|y_2| = |x_2|$, $\text{nth}(y, i) \approx a$, $\text{nth}(y_1, i) \approx a$, plus the original constraints.



Details

1. Every step relies on the previous steps
2. No inconsistencies thanks to normal forms, weak equivalence graph, equivalence class
3. All of these are consistent with one another
4. In the paper: well-definedness

An Example of Lemma Logs

```
12 (lemma STRINGS_REDUCTION (=> true (and (i
   te (and (>= i 0) (> (str.len x) i)) (and
   (= lsym_3 (str.++ sspre_4 (seq.unit 5) ss
   sufr_5)) (= x (str.++ sspre_4 ssubstr_6 s
   ssufr_5)) (= (str.len sspre_4) i) (= (str
   .len (seq.unit 5)) (str.len ssubstr_6)))
   (= lsym_3 x)) (= (str.update x i (seq.uni
   t 5)) lsym_3))))
```

BASE: **36** lemmas generated

$$
\begin{aligned}
&\mathit{lsym_3} = \mathit{update}(x, i, \mathit{unit}(5)) \wedge \\
&(\text{ITE } 0 \leqslant i < |x|, \\
&\mathit{lsym_3} = \mathit{sspre_4} +\!\!+ \mathit{unit}(5) +\!\!+ \mathit{ssufr_5} \wedge \\
&x = \mathit{sspre_4} +\!\!+ \mathit{ssubstr_6} +\!\!+ \mathit{sssufr_5} \wedge \\
&|\mathit{sspre_4}| = i \wedge \\
&|\mathit{unit}(5)| = |\mathit{ssubstr_6}|, \\
&\mathit{lsym_3} = x\ )
\end{aligned}
$$

R-Update

$$
\frac{y \approx \text{update}(x, i, z) \in \mathsf{S}}{\begin{array}{c}\mathsf{A} := \mathsf{A}, i < 0 \vee i \geqslant \ell_y \qquad \mathsf{S} := \mathsf{S}, x \approx y \quad \| \\ \mathsf{A} := \mathsf{A}, 0 \leqslant i < \ell_y, \ell_k \approx i, \ell_{k'} \approx 1 \\ \mathsf{S} := \mathsf{S}, y \approx k +\!\!+ k' +\!\!+ k'', x \approx k +\!\!+ \text{unit}(z) +\!\!+ k''\end{array}}
$$

```
13 (lemma STRINGS_ARRAY_NTH_TERM_FROM_U
   PDATE (=> (= (str.update x i (seq.un
   it 5)) lsym_3) (= (seq.nth lsym_3 i)
    (ite (and (>= i 0) (< i (str.len x)
   )) (seq.nth (seq.unit 5) 0) (seq.nth
    x i)))))
```

EXT: **22** lemmas generated

$$
\begin{aligned}
&\mathit{lsym_3} = \mathit{update}(x, i, \mathit{unit}(5)) \rightarrow \\
&\mathit{nth}(\mathit{lysm_3}, i) = \\
&(\text{ITE } 0 \leqslant i < |x|, \text{nth(unit(5), 0), nth(x, i) })
\end{aligned}
$$

Nth-Update

$$
\frac{\begin{array}{c}\text{nth}(x, j) \in \mathcal{T}(\mathsf{S}) \\ y \approx \text{update}(z, i, v) \in \mathsf{S} \qquad \mathsf{S} \models x \approx y \text{ or } \mathsf{S} \models x \approx z\end{array}}{\begin{array}{c}\mathsf{A} := \mathsf{A}, j < 0 \vee j \geqslant \ell_x \quad \| \\ \mathsf{A} := \mathsf{A}, i \approx j, 0 \leqslant j < \ell_x \qquad \mathsf{S} := \mathsf{S}, \text{nth}(y, j) \approx v \quad \| \\ \mathsf{A} := \mathsf{A}, i \not\approx j, 0 \leqslant j < \ell_x \qquad \mathsf{S} := \mathsf{S}, \text{nth}(y, j) \approx \text{nth}(z, j)\end{array}}
$$